

Compiler Vector Investigation

Algo $\rightarrow$ runtime



Algo $\rightarrow$ Compiler $\rightarrow$ Machine-code $\rightarrow$ Runtime

$\swarrow$
investigate for increasing
Performance
Opportunity.

To generate Assembly: $\rightarrow$

```
g++ -O3 -std=c++20 -S GEMM.cpp -o GEMM.s
```

$\downarrow$
aggressively
optimize

$\downarrow$
stop at
Assembly.

SIMD $\Rightarrow$ Single Instruction Multiple

↓ Data
One instruction on Multiple Data.

Scalar → one float at Time

ex `mulss`
`addss`

Packed :→ Can be processed
Simultaneously

Registers

XMM : 128 bit SIMD register

↳ 4 float Together

YMM → 256 AVX register

↳ 8 float. Together.

AVX VS SSE

YMM

8 floats

at once

XMM registers

4 floats

at once.

in Codebase : Gemm.c/h

Till date June 2

We Did →

```
grep -i xmm GEMM.s
grep -i ymm GEMM.s
grep -i vfmadd GEMM.s
grep -i mulss GEMM.s
grep -i vmul GEMM.s
grep -n "Gemm" GEMM.s
grep -n "_Z4Gemm" GEMM.s
grep -n "Gemm_" GEMM.s
```

We found XMM mulhs

addhs mulss

Not found.

ymm

vmulhs

vfmadd

vs performance.

Hot vs Cold Code



highly executed
Code



Early Executed
Code

↓ apply SIMD here.

Broadcast

Copy one value in entire SIMD
lane

in Gemm $i k j$



ak never changes so

$[ak \dots]$

→ $[ak \ ak \ ak \ ak]$

Broadcasted.

Currently Code is already vectorized
by Compiler. it broadcasted itself

currently ↓

Final Conclusions

GCC Auto-Vectorized Gemm()

Confirmed.

Using:

XMM

mulps

addps

broadcast

GCC Auto-Vectorized Gemm_ijk()

Confirmed.

Using SIMD reduction techniques.

GCC Auto-Vectorized Gemm_tiled()

Confirmed.

Very similar to Gemm().

No AVX

No:

ymm

found.

No FMA

NO FMA

No:

vfmadd

found